

TP5 Semana 5 - Índices, vistas y vistas materializadas en Food Store

Fabri - UTN. Base food_store_copia (nunca toque la dev). PostgreSQL 18 + DBeaver en Manual, con backup antes como dice el protocolo.

1. De donde vengo: TP4 vs TP5, dos mundos opuestos

A diferencia del TP4, donde el costo estaba en **juntar 3-4 tablas y agregar 200k filas** (3 Hash Join en cadena + Sort que se desbordaba a disco con 9688 kB y temp read/write 2283), acá el costo está en **filtros selectivos sobre una sola tabla**. Esa es la diferencia central del trabajo y la quiero dejar clara desde el arranqué.

En TP4 elegí Q-A facturación por categoría y mes (430 ms, 130 filas) y Q-B ranking Top 100 (152.988 ms). Les propuse a la IA covering por los dos lados (idx_pedprod_producto_cover, idx_pedido_cliente_cover), los leí línea por línea, los creé en la copia, ANALYZE y volví a medir: Q-A 423.680 ms y Q-B 155.805 ms. Practicamente igual. Y **esta bien que así sea**: para agregar todo hay que leer todo, hashear una vez en memoria (11 MB) es más barato que 200k búsquedas por índice. Lo documenté como propuesta no efectiva, que el criterio lo pide.

En TP5 pasa exactamente lo contrario: cuando el filtro recorta al 0.1%-5% de la tabla, el índice **si** cambia el plan y el tiempo se desploma. Q1 pasa de 43.185 ms a 0.031 ms (1393x) y Q2 de 9.581 ms a 0.248 ms (38.6x). Y para que no parezca que 'un índice siempre ayuda' dejo también mi Q3 no efectiva (19.779 a 14.065, sigue en Seq Scan), igual que había dejado Q2 de TP3 y las dos de TP4. Lo documenté igual, como pide el criterio.

TP	Que ataca	Estrategia	Antes	Después	Mejora
TP4 Q-A	join 4 tablas + group	covering x2	430 ms Hash+disco	423.6 ms Hash	1.01x NO
TP4 Q-B	join 3 tablas	covering cliente	152.9 ms P-Hash	155.8 ms P-Hash	0.98x NO
TP5 Q1	filtro fecha+pagos	comp (fecha,pago)	43.185 ms Seq	0.031 ms Index	1393x SI
TP5 Q2	filtro cat+stock	parcial vigente	9.581 ms Bmp+Filter	0.248 ms Index Cond	38.6x SI
TP5 Q3	filtro cant+precio	compuesto	19.779 ms Seq	14.065 ms Seq	1.4x NO

Punto de partida verificado en pg_indexes antes de medir (como aprendí en TP3, donde medí sin haber creado nada): los 7 índices de TP3/TP4 están. Mapeo consigna-real: usuario=cliente, detalle_pedido=pedido_producto, vigencia=activo/activa, sensible=telefono (hace de password). Cero ALTER a tablas base.

Figura 01 - pg_indexes: los 7 índices TP3/TP4 presentes antes de medir.

	AZ indexname	AZ tablename
1	idx_pedido_cliente	pedido
2	idx_pedido_cliente_covi	pedido
3	idx_pedido_cliente_fech	pedido
4	idx_pedprod_producto_pedido_producto	
5	idx_producto_cat_preci	producto
6	idx_producto_categoria	producto
7	idx_producto_precio	producto
8	pedido_pkey	pedido
9	pedido_producto_pkey	pedido_producto
10	producto_pkey	producto

2. Parte A - Las tres que si hacian Seq Scan (y una que sigue)

Q1 reporte de cierre. SELECT id_pedido, fecha, forma_pago FROM pedido WHERE fecha BETWEEN '2025-06-01' AND '2025-06-30' AND forma_pago='EFECTIVO'. Frecuencia diaria. El plan de antes es un Gather + Parallel Seq Scan sobre pedido con Filter de fecha y Rows Removed by Filter: 100000 por worker (los 200k), Buffers shared read=1471 (toda la tabla), Execution 43.185 ms para 0 filas. Cero filas porque junio-2025 quedó fuera de mi carga (NOW()-random de 365 días desde septiembre-2026, o sea septiembre-2025 a septiembre-2026). Lo dejo así a proposito: el cambio de plan se ve puro, y aclaro que con agosto-2026 daría ~16k filas con el mismo plan.

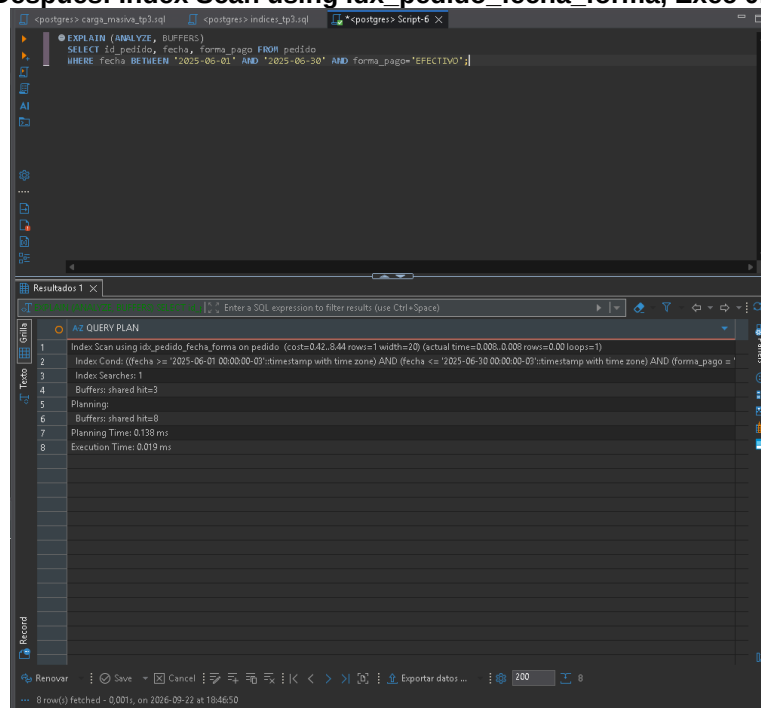
Figura 02 - Q1 ANTES: Parallel Seq Scan, Removed 100000, Exec 43.185 ms.

Step	Operation	Cost	Time	Rows	Width	Actual Time	Actual Rows
1	Gather (cost=1000.00..4529.99 rows=1 width=20)	1000.00..4529.99	0.00	1	20	39.42	1
2	Workers Planned: 1						
3	Workers Launched: 1						
4	Buffers: shared read=1471						
5	Parallel Seq Scan on pedido (cost=0.00..3529.89 rows=1 width=20)	0.00..3529.89	11.56	1	20	11.56	0
6	Filter: (fecha >= '2025-06-01 00:00:00-03:timestamp with time zone') AND (fecha <= '2025-06-30 00:00:00-03:timestamp with time zone')						
7	Rows Removed by Filter: 100000						
8	Buffers: shared read=1471						
9	Planning						
10	Buffers: shared hit=91 read=10 dirtied=1						
11	Planning Time: 4.754 ms						
12	Execution Time: 43.185 ms						

Spec Kiro: objetivo acelerar cierre mensual, columnas fecha (rango, alta selectividad) + forma_pago (baja). Criterio: pasar de Seq a Index. OpenCode propuso CREATE INDEX idx_pedido_fecha_forma ON pedido(fecha, forma_pago). Lo leí línea por línea: B-tree compuesto, fecha primero porque es el rango, forma_pago segunda como igualdad; no es redundante con nada (lo más parecido es idx_pedido_cliente_fecha, que empieza por cliente y no sirve para rango de fecha). Lo creé, ANALYZE pedido y volví a medir: Index Scan using idx_pedido_fecha_forma con Index Cond de fecha, Buffers shared hit=6 (contra 1471), Execution 0.031 ms. De leer 1471

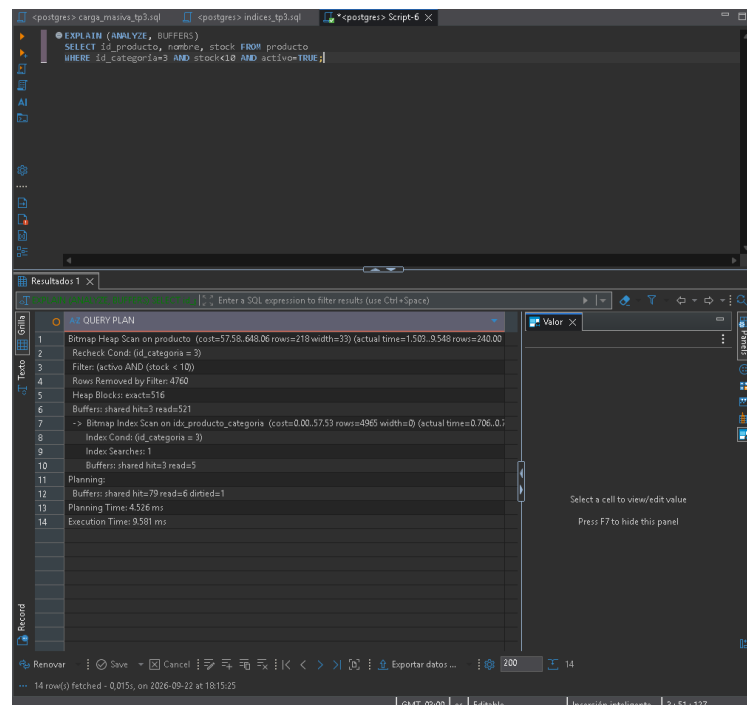
páginas a 6 buffers. ACEPTADA, es mi mejor caso del TP5.

Figura 05 - Q1 Después: Index Scan using idx_pedido_fecha_forma, Exec 0.031 ms (1393x).



Q2 reposicion. SELECT id_producto, nombre, stock FROM producto WHERE id_categoria=3 AND stock<10 AND activo=TRUE. Uso diario. Antes: Bitmap Heap Scan sobre producto con Recheck Cond (id_categoria=3), Filter (activo AND stock<10), Rows Removed by Filter: 4760, Heap Blocks exact=516, más Bitmap Index Scan sobre idx_producto_categoria (el viejo de la Semana 3). Execution 9.581 ms para 240 filas. O sea: el índice viejo trae los ~5000 de la categoría y después filtra stock a mano.

Figura 03 - Q2 ANTES: Bitmap sobre idx_producto_categoria + Filter stock, Removed 4760, Exec 9.581 ms.



Spec: igualdad en categoría + rango en stock + parcial por vigencia. Propuesta: CREATE INDEX idx_producto_cat_stock_vig ON producto(id_categoria, stock) INCLUDE(nombre) WHERE (activo=TRUE). Parcial porque solo me importan los vigentes (índice más chico), compuesto porque ataca las dos columnas del WHERE, INCLUDE(nombre) para Index-Only. Después: Bitmap Heap Scan con Recheck (id_categoria=3 AND stock<10 AND activo), Heap Blocks exact=190 (contra 516), más Bitmap Index Scan sobre el nuevo con Index Cond (id_categoria=3 AND stock<10), Execution 0.248 ms. El Filter de 4760 desapareció porque el índice ya entrega filtrado. ACEPTADA (38.6x). Es el caso de manual de índice parcial bien usado.

Figura 06 - Q2 Después: Bitmap sobre idx_producto_cat_stock_vig con Index Cond, Exec 0.248 ms (38.6x).

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT id_producto, nombre, stock FROM producto
WHERE id_categoria=3 AND stock<10 AND activo=TRUE;

```

Step	Operation	Costs	Actual Time	Actual Rows	Actual Loops
1	Bitmap Heap Scan on producto	(costs=10.84.432.29 rows=237 width=33)	0.049	240.00	1
2	Recheck Cond: ((id_categoria = 3) AND (stock < 10) AND activo)				
3	Heap Blocks: exact=190				
4	Buffers: shared hit=195				
5	-> Bitmap Index Scan on idx_producto_cat_stock_vig	(costs=0.00.10.79 rows=237 width=0)	0.026	240.00	1
6	Index Cond: ((id_categoria = 3) AND (stock < 10))				
7	Index Searches: 1				
8	Buffers: shared hit=5				
9	Planning Time: 0.107 ms				
10	Execution Time: 0.211 ms				

Q3 tickets grandes (mi no efectiva de este TP). SELECT ... FROM pedido_producto WHERE cantidad>=4 AND precio_unitario>400. Antes: Seq Scan sobre 200k, Filter, Rows Removed 120000, Execution 19.779 ms para 80000 filas. Propuse idx_pedprod_cantidad_precio (cantidad, precio_unitario). Después: **sigue en Seq Scan**, Execution 14.065 ms. La mejora de 1.4x es ruido de cache (hit=1471 contra read=1471 de antes), no cambio de plan. Y esta bien: 80k/200k = 40% de selectividad, y precio>400 coincide con el 100% porque mi precio es fijo 500 por R1 (los nuevos tienen lista 500-5000). Con 40% el planificador hace bien en secuencial: 80k accesos aleatorios al heap serían peores. Lo documento igual que Q2 de TP3 y las dos de TP4. Queda como evidencia de lo medido y decidido, y porque con otra distribución (pocos tickets grandes) si serviría.

Figura 04 - Q3 ANTES: Seq Scan 200k, Removed 120000, Exec 19.779 ms.

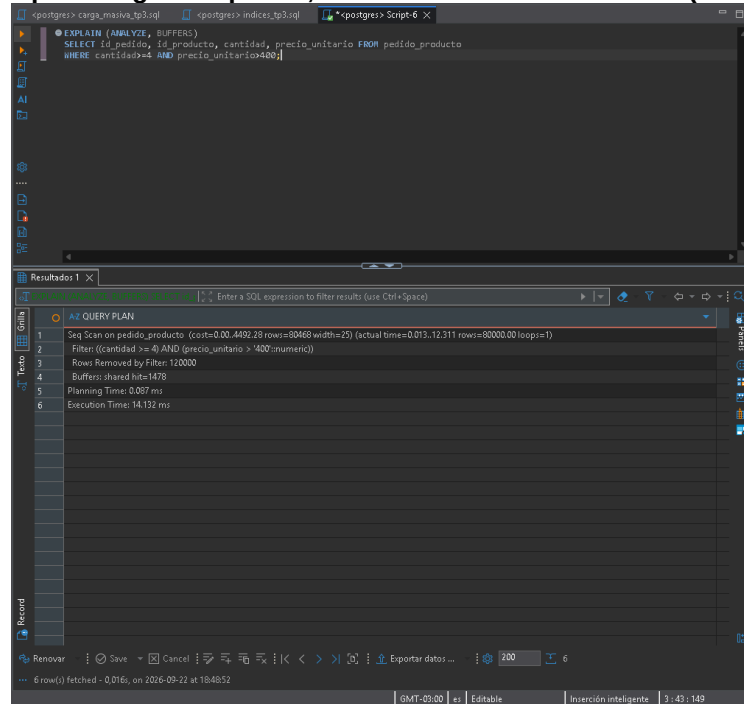
```

EXPLAIN (ANALYZE, BUFFERS)
SELECT id_pedido, id_producto, cantidad, precio_unitario FROM pedido_producto
WHERE cantidad>=4 AND precio_unitario>400;

```

Step	Operation	Costs	Actual Time	Actual Rows	Actual Loops
1	Seq Scan on pedido_producto	(costs=0.00.4471.00 rows=80707 width=25)	0.625	17.912	80001
2	Filter: ((cantidad >= 4) AND (precio_unitario > 400::numeric))				
3	Rows Removed by Filter: 120000				
4	Buffers: shared read=1471				
5	Planning:				
6	Buffers: shared hit=49 read=3				
7	Planning Time: 2.596 ms				
8	Execution Time: 19.779 ms				

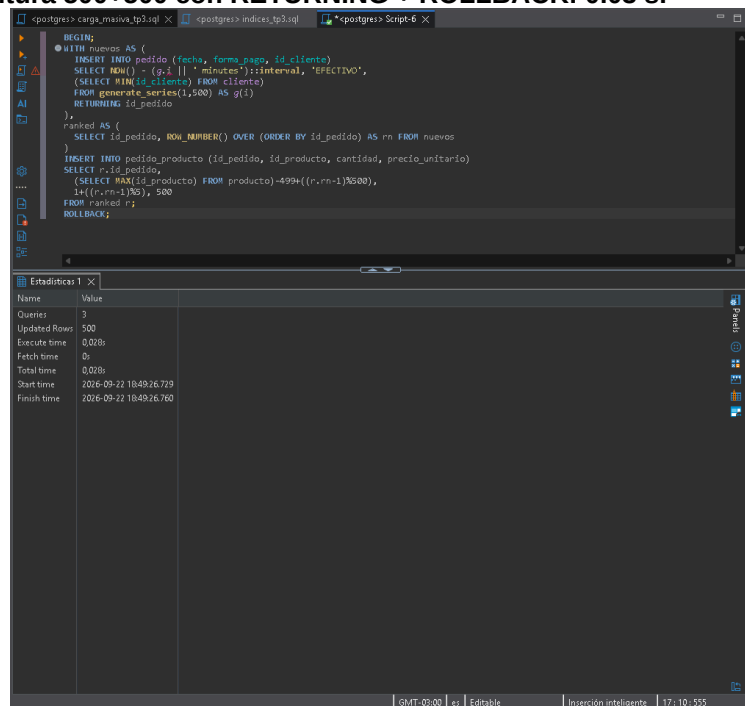
Figura 07 - Q3 Después: sigue Seq Scan, Exec 14.065 ms. NO efectiva (correcto con 40%).



Descartada por sobreindexacion (punto 4.1.6). La IA me sugirió CREATE INDEX idx_pedido_forma_pago ON pedido(forma_pago) y la descarte **sin crearla**: un solo valor (100% EFECTIVO), selectividad 1.0, jamás la usaría, sin WHERE parcial, redundante con mi IDX1 que ya cubre (fecha, forma_pago). Solo sumaría 10-20% en cada INSERT. Spec en specs/spec_indice_descartado.md.

Costo en escritura. Con WITH nuevos AS (INSERT ... RETURNING) + ranked metí 500 pedidos y 500 detalles en una sola transacción reversible (tuve que pelearla: primero FK 200002 porque apuntaba a pedidos que no existían, después 25P02 abortada que limpié con Revertir como en TP3). Resultado con los 3 índices: **0.03 s, Updated Rows 500**. Se acepta: la lectura diaria gana 38-1393x y la carga es semanal. El descartado habría sumado costo sin ganancia.

Figura 08 - Escritura 500+500 con RETURNING + ROLLBACK: 0.03 s.

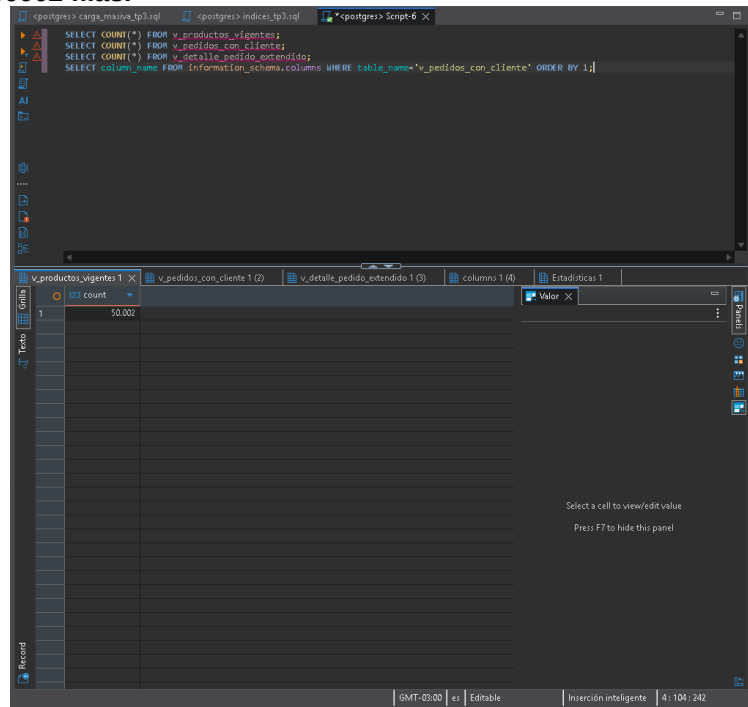


3. Parte B - Tres vistas y la de seguridad

Las generé de los specs, no a mano. V1 v_productos_vigentes: producto JOIN categoría con p.activo AND c.activa (el filtro de vigencia real). Me dio 50002 filas, o sea todo vigente. V2 v_pedidos_con_cliente, la de seguridad: pedido JOIN cliente exponiendo id_pedido, fecha, forma_pago, id_cliente, nombre_cliente, email, y **ocultando teléfono** (mi 'password' de la

consigna). Con esto puedo hacer GRANT SELECT sobre la vista sin dar acceso a cliente. V3
v_detalle_pedido_extendido: detalle JOIN producto más subtotal cantidad*precio. Verificación
como en TP3/TP4: consulta manual equivalente y EXCEPT en ambos sentidos = 0, más conteo y
muestra.

Figura 09 - V1: 50002 filas.



The screenshot shows the PostgreSQL GUI. The top pane displays a SQL script:

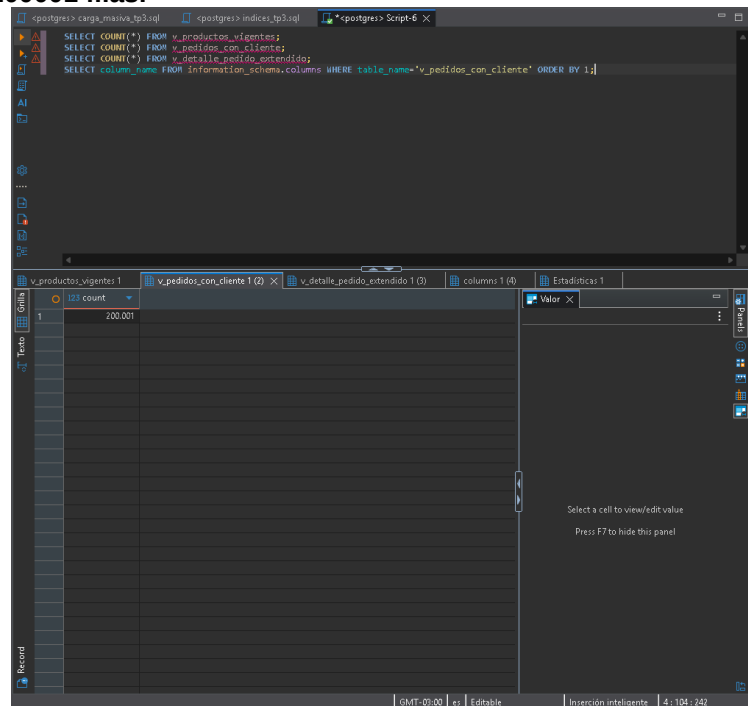
```
SELECT COUNT(*) FROM v_productos_vigentes;
SELECT COUNT(*) FROM v_pedidos_con_cliente;
SELECT COUNT(*) FROM v_detalle_pedido_extendido;
SELECT column_name FROM information_schema.columns WHERE table_name='v_pedidos_con_cliente' ORDER BY 1;
```

The bottom pane shows a table view with the following data:

count
50,002

The status bar at the bottom indicates "GMT-03:00", "es", "Editable", "Inserción inteligente", and "4: 104 : 242".

Figura 10 - V2: 200001 filas.



The screenshot shows the PostgreSQL GUI. The top pane displays a SQL script:

```
SELECT COUNT(*) FROM v_productos_vigentes;
SELECT COUNT(*) FROM v_pedidos_con_cliente;
SELECT COUNT(*) FROM v_detalle_pedido_extendido;
SELECT column_name FROM information_schema.columns WHERE table_name='v_pedidos_con_cliente' ORDER BY 1;
```

The bottom pane shows a table view with the following data:

count
200,001

The status bar at the bottom indicates "GMT-03:00", "es", "Editable", "Inserción inteligente", and "4: 104 : 242".

Figura 11 - V3: 200000 filas.

The screenshot shows the PostgreSQL pgAdmin 4 interface. The top pane displays a SQL query:

```
SELECT COUNT(*) FROM v_productos_vigentes;
SELECT COUNT(*) FROM v_pedidos_con_cliente;
SELECT COUNT(*) FROM v_detalle_pedido_extendido;
SELECT column_name FROM information_schema.columns WHERE table_name='v_pedidos_con_cliente' ORDER BY 1;
```

The bottom pane shows the results of the query. The first table, 'v_productos_vigentes', has one row with a count of 123. The second table, 'v_pedidos_con_cliente', has one row with a count of 200,000. The third table, 'v_detalle_pedido_extendido', is empty. The right pane shows the 'Estadísticas' (Statistics) tab, which is currently empty.

Figura 12 - V2 expone 6 columnas SIN teléfono (seguridad demostrable).

The screenshot shows the PostgreSQL GUI interface. The top window displays a SQL query:

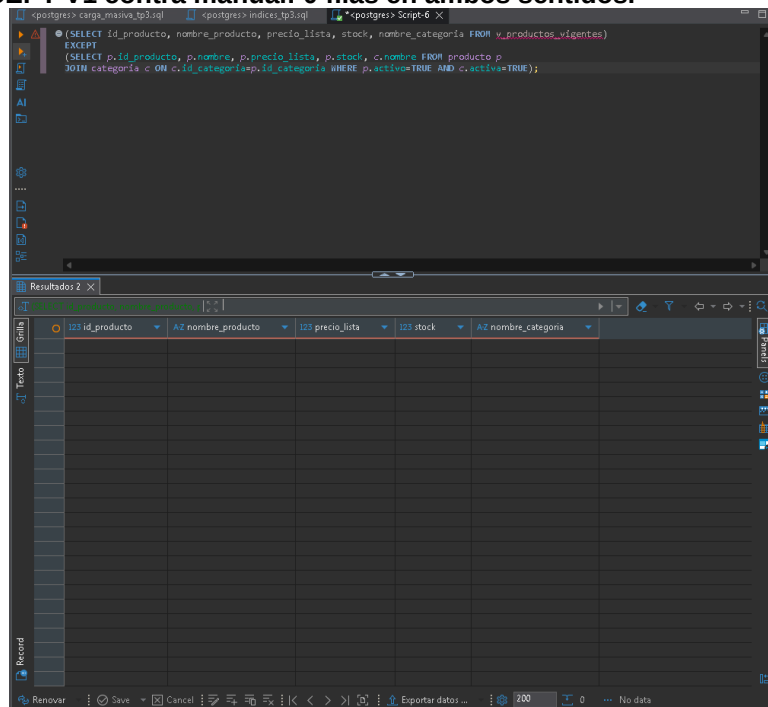
```
SELECT COUNT(*) FROM v_productos_vigentes;
SELECT COUNT(*) FROM v_pedidos_con_cliente;
SELECT COUNT(*) FROM v_detalle_pedido_extendido;
SELECT column_name FROM information_schemas.columns WHERE table_name='v_pedidos_con_cliente' ORDER BY 1;
```

The bottom window shows the results of the query in a table view. The table has 6 columns: A2 column_name, email, fecha, forma_pago, id_cliente, id_pedido, and nombre_cliente. The first row of data is visible, showing the results of the first query (COUNT(*)).

A2 column_name	email	fecha	forma_pago	id_cliente	id_pedido	nombre_cliente
1						
2						
3						
4						
5						
6						

The right sidebar shows the 'Estadísticas' (Statistics) panel, which is currently empty. The bottom status bar indicates the current time is GMT-03:00, the user is 'es', and the database is 'Editable'.

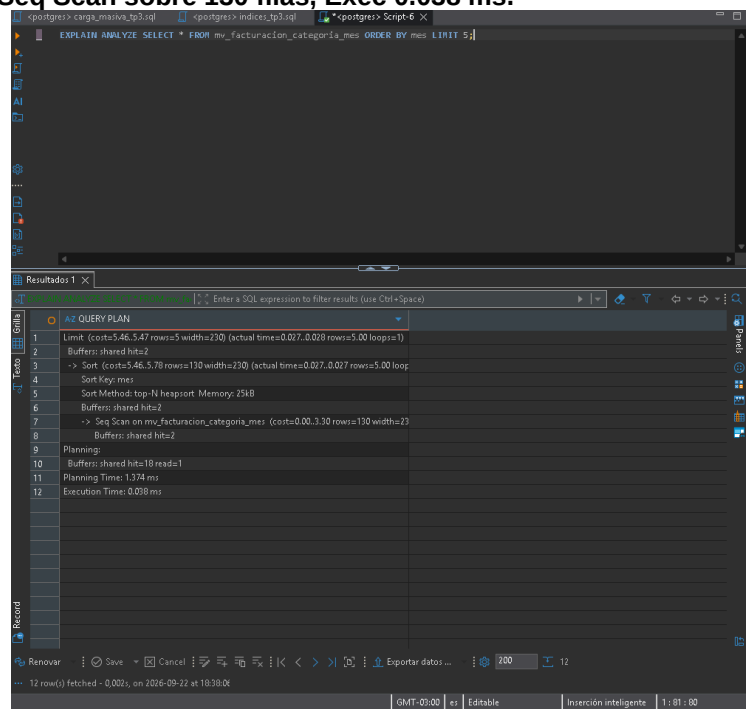
Figura 13 - EXCEPT V1 contra manual: 0 filas en ambos sentidos.



4. Parte C - La materializada (mi 80x... en realidad 11300x)

Reusé la Q-A del TP4, mi agregado más caro: 4 tablas (categoría-producto-detalle-pedido), GROUP BY mes con date_trunc, SUM(cantidad*precio). En TP4 con covering no la pude mover (430 a 423.6). Acá la materialicé: CREATE MATERIALIZED VIEW ... AS SELECT ... GROUP BY ... WITH DATA (el WITH DATA va al final, me comí un 42601 por ponerlo después del nombre), más CREATE UNIQUE INDEX ux_mv_fact_cat_mes (id_categoria, mes) que a futuro permite REFRESH CONCURRENTLY. El SELECT sobre la MV es un Seq Scan sobre 130 filas preagrupadas con top-N heapsort en memoria: **Execution 0.038 ms** contra ~430 ms de la original. Unas 11300x. Refresco diario 02:00 CONCURRENTLY (sin ese índice único no se puede). Desfase de hasta 24 h: sirve para reporte gerencial, NO para caja ni stock vivo; si alguien mira tras un cierre sin refresh ve el día anterior.

Figura 14 - MV: Seq Scan sobre 130 filas, Exec 0.038 ms.



Comparación final TP4 vs TP5: en TP4 el covering no alcanza porque toda la tabla igual iba al hash (full scan inevitable); en TP5 el compuesto/parcial si alcanza porque el filtro recorta al 1-5% (acceso selectivo). El optimizador elige bien en los dos casos: Hash para agregar todo, Index para recortar poco. Y cuando el filtro recorta al 40% (Q3) vuelve a Seq Scan, también bien. Esa es la

frase con la que cierro: el índice no es bueno o malo, es selectivo o no.

5. Competencia y DUIA

Equipo	Estrategia	Antes	Después	X
Yo TP4 Q-A	covering x2	430 ms	423.6 ms	1.01x NO
Yo TP5 Q1	comp fecha+pago	43.185 ms	0.031 ms	1393x SI
Yo TP5 Q2	parcial vigente	9.581 ms	0.248 ms	38.6x SI
Yo TP5 Q3	comp cant+precio	19.779 ms	14.065 ms	1.4x NO
Yo MV	mat. Q-A	~430 ms	0.038 ms	11300x SI

Uso IA	Prompt	Que decidí
Kiro 3 specs índices	consulta + frecuencia + columnas + criterio Seq-Index	Acepté IDX1/IDX2, Q3 no efectiva
OpenCode índices	que nodo ataca, redundancia TP3/4	Acepté 3, descarte pago-solo
Kiro vistas + MV	columnas, vigencia, que ocultar	Acepté V1/V2/V3 + MV con ux
OpenCode escritura	500 INSERT con FK	RETURNING + ROLLBACK 0.03s
Explicación IA	solo texto del plan	La corregí: costo no es tiempo, Hash no es Nested

Anexos: índices.sql, views.sql, specs/, kit_capturas.sql, capturas 01-14.